

Data Engineering Intern Technical Task Report

Devika Rajasekar

1. Part 1 – Data Cleaning

1.1. Entity Relationship Diagram: Figure 1

Objective: The purpose of this ERD is to show how customer and transaction data flows between Salesforce, NetSuite, and Stripe, and how these systems interconnect through shared identifiers most notably, `Stripe_ID`. To design the ERD, I began by analyzing each dataset individually to identify likely primary keys, foreign keys, and attributes critical for linking the systems together. Based on this initial assessment, I developed a conceptual data model and then validated it using a Python script to check the uniqueness and consistency of important fields across datasets.

From my initial visual inspection of the column names and sample rows in each dataset, I assumed that `Account_ID` in Salesforce and `Customer_ID` in NetSuite were the primary keys for those systems. I also noticed that both datasets included a `Stripe_ID` column, which appeared to reference the payment gateway identifier. In the Stripe dataset, I observed that `Customer_ID` potentially matched the `Stripe_ID` values in Salesforce and NetSuite, and that each row seemed to represent a distinct transaction. Based on this structure, I initially hypothesized a one-to-one relationship between Salesforce and NetSuite (via `Stripe_ID`) and a one-to-many relationship from both to Stripe, where Stripe recorded customer-level transactions.

To test these assumptions, I wrote a Python script that performed a series of validation checks:

1. I first checked whether `Account_ID` and `Customer_ID` were unique in their respective systems, to confirm that they could serve as reliable primary keys.
2. Next, I investigated whether `Stripe_ID` values were unique within Salesforce and NetSuite. This revealed that they were not, which challenged my initial assumption of a one-to-one relationship.
3. Lastly, I verified whether a sample `Stripe_ID` was consistent across all three datasets. For the selected value `PAY0713`, the ID was successfully found in Salesforce, NetSuite, and Stripe, confirming that linkage across systems was theoretically valid for at least some records.

These programmatic checks helped refine the ERD by exposing inconsistencies in important relationships especially duplicate `Stripe_ID`s that were not evident from visual inspection alone.

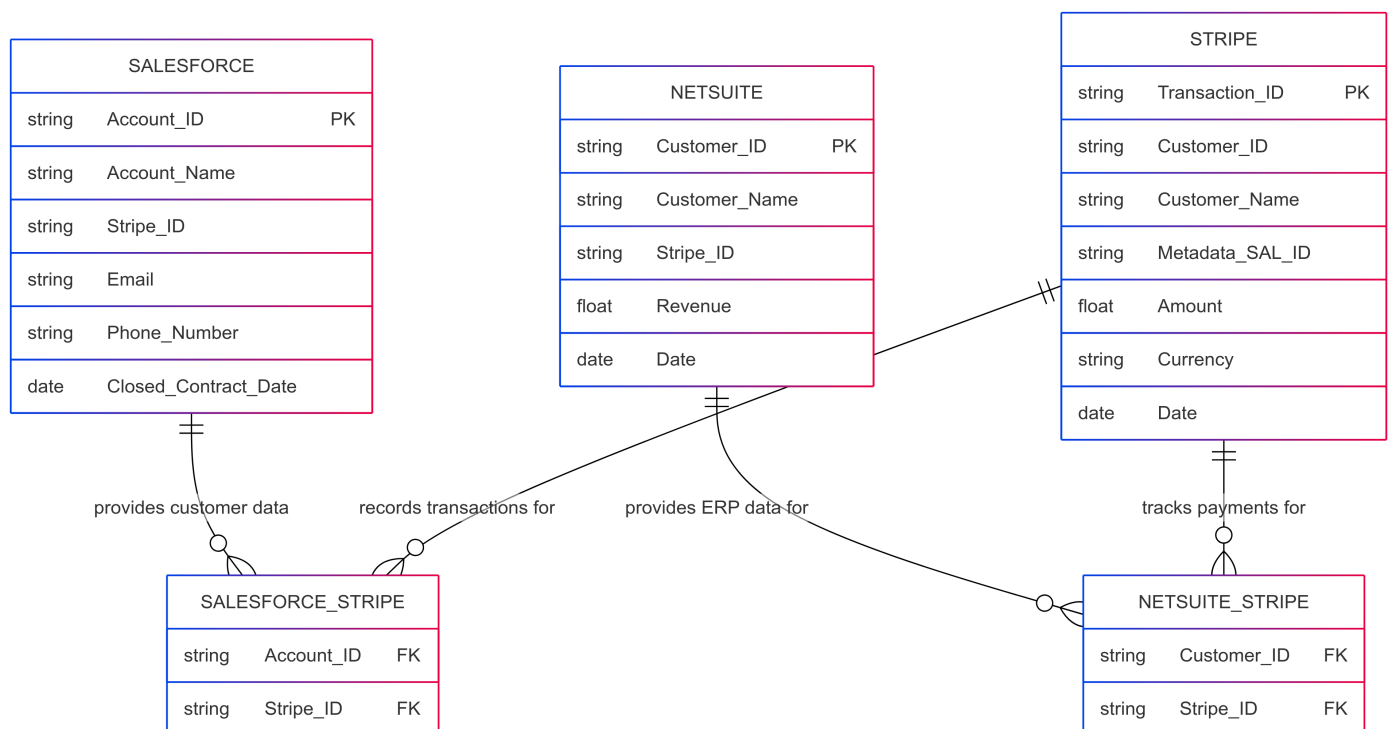


Figure 1. Entity Relationship Diagram

The ERD shown in Figure 1 captures the structure and connections between the three datasets. Each system contains a distinct identifier for customers `Account_ID` in Salesforce, `Customer_ID` in NetSuite, and `Customer_ID` in Stripe with `Stripe_ID` serving as the shared linkage field across systems. To represent many-to-many relationships introduced by duplicate `Stripe_ID`s, I included two junction tables: `SALESFORCE_STRIPE` and `NETSUITE_STRIPE`.

I also introduced a surrogate key, `Transaction_ID`, into the Stripe dataset. This addition was necessary because the dataset originally lacked a unique identifier for each transaction. While it's possible to form a composite key from fields like `Customer_ID`, `Amount`, and `Date`,

this method would be error-prone and not scalable. The surrogate key guarantees that each transaction is uniquely identifiable, simplifies joins across systems, and supports normalization.

From	To	Relationship Type
Salesforce	Stripe	One-to-Many (via Stripe_ID)
NetSuite	Stripe	One-to-Many (via Stripe_ID)
Salesforce	NetSuite	Many-to-Many (via shared Stripe_ID)
Stripe	Transactions	One-to-One (via Transaction_ID)

Table 1. Cardinality of Relationships in the ERD

1.2. Data Cleaning

For accurate integration and analysis, I wrote a Python script that applied systematic cleaning to each dataset. My cleaning pipeline included standard data quality procedures such as deduplication, date parsing, ID standardization, numeric sanitization, and contact field validation. Additionally, I incorporated cross-system checks to identify conflicting records across Salesforce and NetSuite using shared `Stripe_ID`s.

Specifically, the cleaning process involved:

- Converting key identifiers to strings and stripping whitespace to ensure join consistency.
- Removing duplicate rows from each dataset to eliminate redundancy.
- Parsing all date fields (`Date`, `Closed_Contract_Date`) and dropping rows with invalid date entries.
- Cleaning numeric fields (`Revenue` in NetSuite, `Amount` in Stripe) by removing non-numeric characters, converting to numeric type, and filtering out extreme outliers (values > 1 million).
- Validating email addresses in Salesforce using regex and flagging invalid entries.
- Sanitizing phone numbers by removing non-digit characters.
- Standardizing customer and account names by stripping extra spaces and applying title casing.
- Performing a cross-dataset merge on `Stripe_ID` to identify naming mismatches between Salesforce and NetSuite records.

Cleaning Report and Interpretation: After cleaning, I generated a summary report to review the transformations and data health across the three systems:

- In the **NetSuite** dataset, 387 valid rows remained after dropping 643 duplicates and filtering out invalid revenue or date records. The dataset had no missing `Stripe_ID`s, indicating good referential coverage.
- In **Salesforce**, the dataset retained 1,000 unique and valid rows after eliminating 30 duplicates. All email addresses were valid, phone numbers were correctly formatted, and there were no missing or invalid contract dates.
- The **Stripe** dataset was the largest and required the most reduction: 3,333 transaction rows were retained after removing 7,173 duplicates. All currency values were valid, and the transactions spanned multiple currencies including EUR and USD.

Finally, a cross-system name conflict check between Salesforce and NetSuite revealed **zero naming mismatches** for customers sharing the same `Stripe_ID`, which was a positive sign for record alignment. This result increased my confidence in using `Stripe_ID` as a linkage field, while the success of the cleaning process overall helped ensure data integrity for downstream integration and modeling.

2. Part 2 – Data Integration & Analysis

The goal of this phase was to understand how customer records connect across the three given systems: Salesforce (CRM), NetSuite (ERP), and Stripe (Payment Gateway). I aimed to assess whether customers and transactions could be reliably matched across these platforms and to surface any inconsistencies that might hinder data integrity or reporting accuracy.

1. Customer Behind the Largest Payment:

- I began by examining the Stripe transaction data, ensuring that all `Amount` fields were correctly parsed as numeric values. This step was needed to prevent skewed results due to formatting errors or any missing values. Any invalid or missing payment amounts were removed.
- Then, I identified the single largest payment in the Stripe dataset. Using the `Customer_ID` (which serves as the Payment Gateway ID), I linked this record to Salesforce using the `Stripe_ID` field.
- **Finding:** The customer associated with the largest payment had the email address `alexjohnson@example.com`.

2. Most Active Customer:

- I grouped all Stripe transactions by `Customer_ID` to count the number of payments made by each customer.
- To focus on known entities, I filtered the results to include only those `Customer_ID`s that also existed in NetSuite. This ensured that only valid ERP-recognized customers were included in the analysis.
- The most active customer (i.e. the one with the highest transaction count) was then matched to Salesforce via `Stripe_ID` to retrieve their CRM profile.
- **Finding:** The most active customer was:
 - **Account ID:** SAL0651
 - **Account Name:** Morgan Smith
 - **Stripe ID:** PAY0989
 - **Email:** morgansmith@example.com
 - **Phone Number:** 225315720088
 - **Closed Contract Date:** 2024-11-08

3. Merging Financials from ERP and Stripe:

- To gain a more holistic financial view per customer, I aggregated total revenue from NetSuite and total cash inflows from Stripe.
- Both datasets were grouped by their shared Payment Gateway ID (Stripe_ID in NetSuite and Customer_ID in Stripe).
- Then, I merged the two aggregates to create a consolidated financial profile per customer.
- The merged dataset was saved as a CSV file. The final dataset consisted of 976 unique customers, each with their corresponding revenue and Stripe payment totals.

Table 2. Sample Financial Metrics per Customer

Payment Gateway ID	Total Revenue (NetSuite)	Total Cash (Stripe)
PAY0001	–	2,536.15
PAY0002	1,206.65	91,206.79
PAY0003	–	847.40
PAY0997	2,687.45	2,311.37
PAY1000	3,312.34	4,745.23

Note: "–" indicates missing data in NetSuite.

- One notable insight was that several customers had significant payment volumes in Stripe but no corresponding revenue in NetSuite. This observation warranted further investigation into whether these customers were properly onboarded in the ERP system.

4. Data Linking Issues:

- I conducted additional checks to identify any gaps in cross-system linkage. Specifically, I searched for records that were missing Stripe_ID values in either Salesforce or NetSuite, which would prevent any join with Stripe.
- I also checked for Stripe transactions whose Customer_IDs did not exist in the NetSuite dataset.
- **Findings:**
 - (a) 24 customer records in NetSuite were missing Stripe_IDs entirely, making them unmatchable to any payment data in Stripe.
 - (b) 2,091 Stripe transaction records had Customer_IDs that did not match any entry in NetSuite, suggesting a potential breakdown in customer onboarding or integration syncing.
- These mismatches show weak points in the integration pipeline and represent a risk to data completeness and reliability in downstream analytics.

This analysis revealed areas for improvement:

- There is a need for stricter validation at the point of customer creation in NetSuite and Salesforce to ensure that Stripe_IDs are always present and consistently formatted.
- A routine synchronization audit such as a nightly job that flags unmatched Stripe_IDs could help detect integration failures early.
- Ultimately, implementing a Master Customer ID system would simplify future integration and ensure long-term consistency across systems.

3. Part 3 – System Improvement

3.1. Improving Data Architecture for Accurate Matching

To achieve 100% accurate and scalable matching of customer records across Salesforce (CRM), NetSuite (ERP), and Stripe (Payment Gateway), the current architecture must evolve beyond fragmented identifiers like Account_ID, Customer_ID, and Stripe_ID. These system-specific IDs are not guaranteed to align and are often inconsistently formatted. In some cases, they may even be missing altogether, which results in unreliable joins and leads to duplicated or orphaned records in analysis.

The foundation of my proposed architecture is the introduction of a centralized Master_Customer_ID. This unique identifier would be generated at the point of customer onboarding and persistently attached to every customer record as it flows across systems. It becomes the authoritative identity key used to reliably link customer data across platforms.

Components of this architecture:

- **Master Identity Service:** A lightweight, cloud-native service (e.g., built with AWS Lambda + API Gateway) that generates and returns a Master_Customer_ID whenever a new customer is registered via UI or API. This makes sure of a uniform starting point for all records.
- **Customer Master Database:** A centralized store (DynamoDB for high availability or Amazon RDS for relational features) that holds mappings between Master_Customer_ID and the corresponding external identifiers from Salesforce, NetSuite, and Stripe. This table ensures traceability and auditability of ID lineage.
- **Event-Driven Distribution:** Upon creation of a new Master ID, an event (via AWS EventBridge) is published and automatically picked up by downstream systems. This design ensures that Salesforce, NetSuite, and Stripe are updated asynchronously, reducing latency and maintaining eventual consistency.
- **Schema and Validation Layer:** All incoming customer and transaction data must pass schema validation and include a valid Master_Customer_ID before being accepted by any target system. This helps ensure data consistency from the start and prevents propagation of mismatches.
- **Downstream Integration:** Each system stores the Master ID in its local schema alongside its native ID. For example, Salesforce would store both Account_ID and Master_Customer_ID which would in turn enable seamless cross-system joins.

3.2. Real-Time QA for Spotting Data Mismatches

While centralizing customer identity solves the matching problem structurally, it's just as critical to build guardrails that validate data linkages continuously. I would suggest establishing an event-driven Quality Assurance (QA) layer that detects mismatches, inconsistencies, and missing values in real-time as data moves through the pipeline.

Why real-time QA matters: Batch jobs or periodic audits can catch errors eventually, but they are often too late to prevent bad data from affecting reports, triggering false alerts, or even damaging customer experiences. Instead, an event-driven Quality Assurance (QA) layer would make sure that validation happens immediately as new records are created or updated.

Core components of the real-time QA pipeline:

- **Event Triggering:** Every time a new customer or transaction is created, an event can be emitted via AWS EventBridge or Kafka. These events contain the payload to be validated. For e.g., customer name, Stripe_ID, Master_Customer_ID, or email address.
- **Validation Microservice:** This service would run a lightweight validation logic derived from the earlier Python validation scripts. It checks:
 - Whether the provided Master_Customer_ID exists in the identity store.
 - Whether the same Stripe_ID maps to conflicting names or emails in different systems.
 - Whether required identifiers like Stripe_ID or Customer_ID are missing or duplicated.
- **Routing Decisions:**
 - If the data passes validation, it is forwarded downstream to its target system (e.g., Salesforce or Stripe).
 - If it fails, the record is either quarantined (held in S3 or a DynamoDB staging table) or flagged for manual review.
- **Alerting and Visibility:** Validation errors trigger real-time alerts using Amazon SNS and are delivered via AWS Chatbot to Slack or other communication channels. Simultaneously, CloudWatch metrics and dashboards could display real-time trends such as the number of unmatched customer records, duplicate IDs, or conflicts per day.

By catching issues at the point of entry, this QA pipeline prevents conflicting data from entering reporting systems. It also provides visibility into the health of cross-system relationships, which would help engineering teams respond quickly.

3.3. Automating QA with AWS Services

To fully industrialize the QA process, I would leverage AWS's broad suite of analytics, monitoring, and automation services. The goal would be to build a resilient pipeline that validates and monitors data at every stage and alerts when problems arise. It would also provide a clear view of system health.

a. Cleaning and Validating Incoming Data

Before data can be joined across systems, it needs to be clean and complete. I would use the following tools to enforce data quality rules right from the start:

- **AWS Glue DataBrew** is a visual, low-code tool that lets us define validation rules (e.g., phone numbers should be digits only, currency fields should follow specific formats). It's ideal for spotting issues without writing complex code.
- **AWS Glue ETL Jobs** would run nightly to combine Salesforce, NetSuite, and Stripe data and generate statistics like "how many customers are missing master IDs" or "how many Stripe transactions have no ERP match." These metrics can be compared day to day to catch regressions.
- **Amazon Athena** would allow us to write SQL-like queries directly on the data stored in Amazon S3. We can quickly check for exceptions like duplicate customer records or support tickets linked to nonexistent accounts.

b. Monitoring Data in Real-Time

Even with nightly validation, some errors need to be caught immediately, especially if they could impact critical reports or customer-facing systems. For this, we would have to rely on real-time event processing:

- **AWS Lambda** is a serverless compute tool that automatically reacts when new data is created. It can instantly run custom validation logic such as checking whether a new Stripe transaction includes a known Master Customer ID.
- **AWS EventBridge** would act as a central nervous system. Whenever a new customer or transaction enters the system, it would trigger validation functions so that issues are flagged the moment they appear.
- **Amazon CloudWatch** would help track the health of these validations. We can define custom metrics like "percentage of unmatched records" or "duplicate Stripe IDs per hour." If a metric crosses a critical threshold, we can respond right away.

c. Alerting and Orchestration

Identifying an issue is only part of the solution - we also need to make sure the right people are notified, and that the response process is organized and efficient.

- **Amazon SNS (Simple Notification Service)** sends alerts via email, SMS, or integrated messaging apps.
- **AWS Chatbot** can send those alerts directly into Slack channels, where data teams can immediately investigate and take action.
- **AWS Step Functions** could help organize the full QA pipeline into an automated sequence. For example, the system can: receive new data → run validation → quarantine bad records → alert the data team all without any manual effort.

d. Maintaining Trust with Governance and Auditing

As we scale our data ecosystem, maintaining control and traceability becomes crucial. These tools help ensure we know where the data came from, whether it's valid, and who changed what.

- **AWS Glue Data Catalog** acts as a central directory for all our datasets. It stores schema definitions and helps enforce rules like “every transaction must have a Master ID.”
- **AWS CloudTrail** logs every interaction with our AWS environment like when someone updates a validation rule or changes the schema providing a full audit trail for compliance and debugging.

4. Part 4 – Expanding Connections

4.1. Additional Data Sources

While integrating the core systems Salesforce (CRM), NetSuite (ERP), and Stripe (Payment Gateway) enabled a unified view of customer records and transactions, it left a critical gap: we still lack insight into how customers experience this service day-to-day. To get a better idea of this, I propose integrating a new source: **support ticket data**.

Why Customer Support Tickets?

Customer support interactions (typically managed in systems like Zendesk, Intercom etc.) are a direct lens into customer sentiment and operational friction. By incorporating support ticket metadata into the data architecture, we gain a fuller picture of the customer journey beyond just transactional and contract data.

Some examples of insights this could enable:

- **Churn prediction:** Customers with high ticket volumes, repeated issues, or long resolution times may be at higher risk of churn. Especially when this is correlated with declining product usage or revenue.
- **Retention and upsell targeting:** High-value customers with consistently positive support interactions could be candidates for testimonials or premium offers.
- **Product improvement:** Aggregated support trends can reveal systemic pain points in onboarding, billing, or feature usage informing product and operations roadmaps.

Support ticket records can be joined with existing Salesforce accounts using email addresses, or with Stripe/NetSuite via mapped customer IDs if the support platform tracks billing details. Once integrated, this dataset enables more holistic dashboards and customer health scores.

Future expansion: In parallel, integrating **product usage logs** from systems like Segment could enhance this view further. By aligning actual feature usage with support engagement and revenue data, we can segment customers into categories such as:

- “Silent churners” (low usage + high support issues),
- “Proactive customers” (high usage + low/no tickets),
- “High-risk accounts” (recent high revenue + support escalation).

4.2. Designing the Support Ticket Data Model

To operationalize the integration of support data, I first defined the schema for a new table: `support_tickets`. This schema is designed to represent metadata about customer support interactions and includes fields that can be joined with existing CRM and ERP systems.

Schema Overview:

- `Ticket_ID` (string) – A unique ticket identifier (e.g., TKT1001).
- `Customer_ID` (string) – Reference to the customer in the NetSuite ERP system (`Customer_ID`).
- `Account_ID` (string) – Reference to the corresponding Salesforce account (`Account_ID`).
- `Created_Date` (date) – Timestamp indicating when the ticket was created.
- `Resolved_Date` (date) – Timestamp for when the issue was resolved (nullable).
- `Status` (categorical) – Ticket state (e.g., Open, Closed, Escalated).
- `Category` (categorical) – Type of issue (e.g., Billing, Product, Technical).
- `CSAT_Score` (integer) – Customer Satisfaction score from 1 to 5.
- `Assigned_Agent` (string) – Name of the support representative handling the case.
- `Priority` (categorical) – Ticket urgency level (Low, Medium, High).

This design enables flexible analysis across dimensions such as issue volume, time to resolution, escalation rates, or satisfaction scores either at the individual customer level or aggregated by revenue tier or usage segment.

4.3. Relating to other systems

The support ticket data is linked to existing systems via `Customer_ID` and `Account_ID`, which already exist in NetSuite and Salesforce respectively. This allows for bi-directional joins such as:

- Joining with Salesforce enables enrichment with CRM attributes (e.g., contact info, contract close dates).
- Joining with NetSuite makes it possible to correlate support behavior with revenue, subscription tier, or historical purchasing patterns.

These identifiers would ideally map to a centralized `Master_Customer_ID`, which would enable us to have seamless integration into our unified analytics environment (for e.g. a data lake or warehouse). For the purposes of simulation, randomly generated `Account_ID`s and `Customer_ID`s were used following the same format conventions as in the original systems (e.g. “SAL0001” for Salesforce, “NET0001” for NetSuite).

4.4. Updating the ERD to Include Customer Support Data

Next, I updated the ERD to incorporate the new data source: the SUPPORT entity. The newly introduced SUPPORT table is directly linked to both Salesforce and NetSuite via foreign keys:

- Account_ID (FK) → SALESFORCE.Account_ID
- Customer_ID (FK) → NETSUITE.Customer_ID

This design reflects the fact that support tickets are typically submitted by CRM contacts (Salesforce) but are often related to ERP-linked customers (NetSuite).

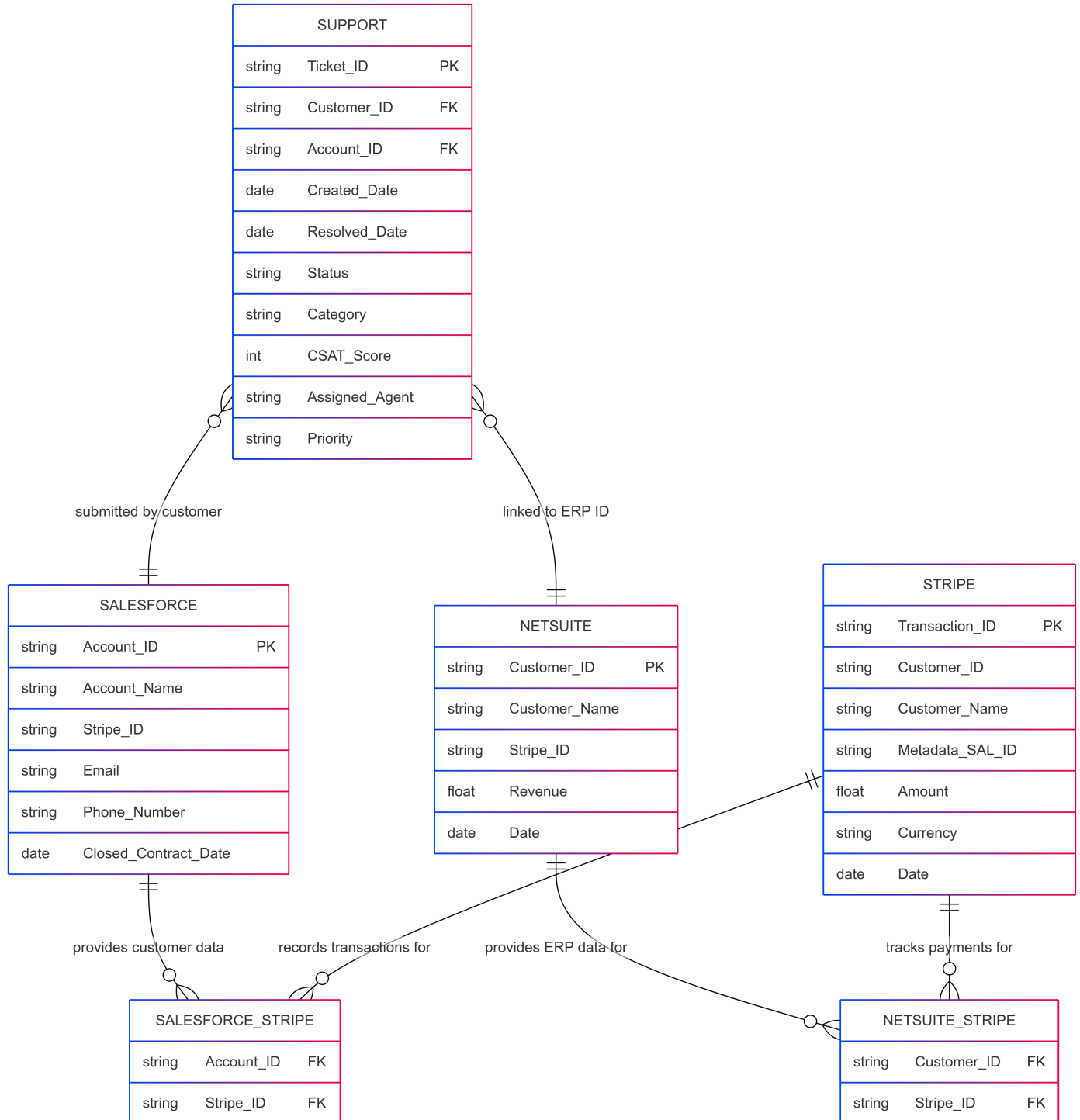


Figure 2. Enhanced ERD

4.5. Mocking the Support Ticket Dataset

To create a realistic mock dataset, I used the `Faker` library in Python, along with the `random` and `datetime` modules to simulate temporal and categorical variability. The dataset generation process included:

- Generating 100 unique Ticket_IDs in sequential order.

- Randomly assigning Salesforce and NetSuite IDs to mimic cross-platform linkage.
- Using `Faker` to create agent names and randomized ticket creation dates within the past 6 months.
- Introducing variability in resolution times (1–15 days), with some tickets left unresolved.
- Randomly assigning status, category, priority level, and customer satisfaction scores to simulate real-world ticket patterns.

The final dataset was exported as a structured JSON file, stored under `data/mock_support_tickets.json`. This file can now be used to test system integrations, validate joins across Salesforce and NetSuite, and support future experiments in churn prediction or customer health scoring.

Example Ticket Record (JSON):

```
{
  "Ticket_ID": "TKT1001",
  "Customer_ID": "NET0587",
  "Account_ID": "SAL0637",
  "Created_Date": "2023-12-02",
  "Resolved_Date": "2023-12-13",
  "Status": "Closed",
  "Category": "Billing",
  "CSAT_Score": 4,
  "Assigned_Agent": "Lucas Jackson",
  "Priority": "High"
}
```

This mock dataset can be extended to include text-based fields (e.g. resolution notes or customer comments) which would enable more advanced use cases like sentiment analysis or natural language classification in the future.